CrossSignal Competitive Research and Final-Submission Enhancement Report

Prepared: 3 September 2026

Scope: Alpaca AI Trading Agents Hackathon submission readiness

Evidence standard: Competition materials supplied with the project plus primary product and platform documentation

Executive decision

> CrossSignal should remain an auditable cross-market options agent. Its strongest differentiator is not a larger collection of indicators or a debate among several language-model agents; it is the conversion of cross-asset disagreement into a sealed, reproducible decision contract, followed by deterministic risk gates and atomic defined-risk execution.

The competitive review found one material product weakness in the prior build: position lifecycle controls existed conceptually but lacked several production safeguards. The current build now adds broker-inventory reconciliation, quote-freshness enforcement, deterministic client order identifiers, an independent new-entry kill switch, and realized lifecycle reporting. These additions directly strengthen the competition requirement to show how the agent manages positions and performs. They do not change the signal thesis.

The remaining competitive weakness is evidence, not architecture. The dedicated account currently shows ten abstentions, zero submitted option legs, zero managed positions, and $0 broker P&L. That is an honest demonstration of risk discipline, but it cannot score strongly in the P&L category. Do not manufacture a trade or relax gates merely to create activity. If time remains, continue scheduled paper cycles and use a real eligible fill and its lifecycle only if the sealed rules authorize it.

Competition objective and judging lens

The supplied 28-page competition capture describes the challenge as building an autonomous agent designed to generate P&L on Alpaca. Entrants must show how the agent identifies opportunities, makes decisions, manages positions, and performs during the competition. The required stack includes Alpaca Trading API access through MCP or CLI, options trading, a fresh $100,000 paper account, a one-page explanation of AI logic and risk gates, and the standard video, slides, repository, demo, and account evidence.

The judging categories are P&L Performance, Technology Implementation, Creativity and Originality, and Presentation and Execution. Therefore, the best final work is a narrow improvement to real execution reliability and judge-visible evidence—not a late expansion into unrelated markets or a new strategy.

Comparable systems and lessons

Alpaca multi-leg options

Alpaca's Level 3 options documentation makes the combined multi-leg order the correct primitive for a spread. It reduces the unbalanced-exposure risk that can occur when legs are submitted independently and supports explicit position intents. CrossSignal already used atomic MLeg entries and exits; the enhancement keeps that structure and makes the lifecycle order idempotent.

Alpaca's orders guidance recommends real-time order updates and supports client order identifiers. CrossSignal now supplies a stable `client_order_id` for each claimed exit attempt and persists the attempt number. A retry can therefore be identified rather than appearing as an unrelated order. A later version should consume `trade_updates` from Alpaca's WebSocket stream for lower-latency fill and rejection reconciliation; scheduled polling remains acceptable for the submission build because it is simpler and already tested.

Alpaca's options overview also highlights expiration, exercise, and liquidation behavior. CrossSignal's sealed pre-expiry rule is therefore not cosmetic: it reduces reliance on broker expiration handling and makes the intended exit auditable.

Option Alpha

Option Alpha's automation documentation separates entry automations from monitors that repeatedly check open positions. Its documented recipes include profit targets, stop losses, expiration checks, and trailing controls. The transferable lesson is a persistent lifecycle with explicit exit criteria—not the wholesale copying of a no-code bot model. CrossSignal now has that lifecycle while retaining its differentiated cross-market thesis and sealed decision receipt.

QuantConnect LEAN

QuantConnect's Algorithm Framework risk-management documentation separates alpha, portfolio construction, risk management, and execution. Risk models can reduce or liquidate targets after portfolio construction. This validates CrossSignal's architectural boundary: Claude proposes a structured thesis, while deterministic code controls authorization, sizing, execution, and exits. CrossSignal should keep this separation and avoid letting generative output directly mutate broker state.

QuantConnect's backtest statistics documentation exposes realized P&L, win/loss counts, drawdown, fees, and risk-adjusted statistics. CrossSignal now reports closed-position count, wins, losses, win rate, realized P&L, average P&L, state counts, and exit reasons. It deliberately does not calculate Sharpe or claim statistical significance with no realized sample.

Alpaca reference agents and operational guidance

Alpaca's NightWatcher V2 case study emphasizes a separation between analysis, policy, execution, and presentation, along with kill switches, loss limits, cooldowns, and approval controls. CrossSignal already had deterministic pre-trade gates; it now adds `PAUSE_NEW_ENTRIES`, which can stop new exposure without disabling legitimate management of positions that already exist.

Alpaca's discussion of automated-trading risks stresses monitoring and alerting for failures. CrossSignal now treats a broker read failure as an error rather than as an empty account, compares registered option-leg exposure with

ledger.

Gap matrix and disposition

Capability	Market precedent	Prior CrossSignal state	Decision	Current result
Atomic multi-leg entry and exit	Alpaca MLeg	Implemented	Keep	Preserved
Persisted take-profit, stop-loss, time, and expiry rules	Option Alpha monitors; QuantConnect risk models	Implemented	Keep	Preserved and judge-visible
Broker inventory reconciliation	Automated-trading operations practice	Missing	Add now	Registered legs are checked against Alpaca before valuation or exit
Stale-quote control	Execution-quality best practice	Missing	Add now	Missing or older-than-policy timestamps block automated exit submission
Idempotent order identity	Alpaca client order identifiers	Missing	Add now	Exit attempts receive deterministic IDs and atomic attempt counters
Manual entry kill switch	Alpaca reference-agent controls	Implicit through deployment flags	Add now	`PAUSE_NEW_ENTRIES` stops entries while lifecycle exits remain available
Realized lifecycle metrics	QuantConnect reporting	Limited	Add now	State counts, realized P&L, win rate, average P&L, and exit reasons
Streaming fill updates	Alpaca WebSocket `trade_updates`	Polling	Defer	Valuable after submission; polling is tested and lower-risk now
Exit-order aging and cancel/reprice	Mature execution systems	Basic terminal-state retry	Defer	Add only with explicit slippage and retry policy tests
Automatic rolling	Options automation platforms	Absent	Reject for this submission	Adds strategy discretion and can increase exposure
More agents, indicators, or asset classes	Various agent demos	Not required	Reject for this submission	Would dilute the cross-market disagreement thesis
Large backtest claims	Quant platforms	Insufficient history	Reject unsupported claims	Report actual paper ledger and clearly labeled replay only

Implemented engineering improvements

The final build now uses named position and exit states instead of scattered magic strings. The lifecycle database migrates existing installations without deleting records. Each exit is reserved through a compare-and-set transition, assigned an incrementing attempt number, submitted with a deterministic client identifier, and reconciled back to the broker result. This is readable, testable, and concurrency-safe.

Before monitoring an open spread, the job fetches Alpaca positions once and reconciles aggregate leg direction and quantity. A transport failure is surfaced, never converted into a false empty-account result. Before an automated close, the monitor verifies that every leg quote contains a broker timestamp and that the oldest quote is within the configured limit. The authoritative market clock and the separate paper-mutation controls still apply.

The reporting layer now distinguishes forecast evaluation from broker execution performance. It exposes lifecycle states and realized figures without implying that illustrative replay data came from Alpaca. The public dashboard labels replay lifecycle metrics as illustrative; the actual performance report currently states that there are no managed positions or realized returns.

Code-quality assessment

The edited code follows a small-module boundary: broker transport remains in `tools/alpaca_tools.py`, deterministic policy and state transitions remain in `agent/position_manager.py`, persistence and aggregation remain in `compliance/audit_logger.py`, and orchestration remains in `live/cross_market_agent.py`. Configuration is environment-driven and validated at startup. Public deployments remain non-mutating by default.

Verification completed on 3 September 2026:

- 59 automated tests passed.
- Python bytecode compilation passed for agent, tools, compliance, live, demo, scripts, app, configuration, and performance reporting.
- The Streamlit application completed a headless smoke test with zero exceptions across seven tabs.
- The real audit database reported ten sealed contracts, all abstentions, no submitted legs, no managed positions, and $0 realized broker P&L.

Submission recommendation

Architecturally, CrossSignal is now submission-ready. The code addresses opportunity identification, structured AI reasoning, deterministic decisions, defined-risk execution, position management, and auditability without drifting from the challenge.

Competitively, submit only after replacing the obsolete video with a new recording that visibly covers the problem, cross-market solution, one authorized replay, one abstention, atomic MLeg construction, broker reconciliation, stale-quote protection, the entry kill switch, the take-profit/stop-loss/time/expiry lifecycle, and the honest zero-trade account state. The presenter should say plainly that no market cycle met every sealed threshold; this justifies the lack of trade but does not claim P&L performance.

If a genuine eligible trade fills before the deadline, capture the Alpaca order, both legs, the registered exit policy, and subsequent monitor event. If none fills, submit the truthful abstention evidence. A forced trade would undermine the strongest part of the project and could create a worse technology and presentation score without producing meaningful performance evidence.

Primary sources

1. Alpaca, Options Level 3 Trading.
2. Alpaca, Working with Orders.
3. Alpaca, WebSocket Streaming.
4. Alpaca, Options Trading Overview.
5. Alpaca, Risks of Automated Trading Systems.
6. Alpaca, Building NightWatcher V2.
7. Option Alpha, Automations.
8. QuantConnect, Algorithm Framework: Risk Management.
9. QuantConnect, Backtest Statistics.

10. Lablab.ai, Alpaca AI Trading Agents Hackathon, cross-checked against the 28-page event capture supplied with this project.